

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour

Total Marks:50

Annexure A

Design Task

Code of Conduct:

I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action

Signature of the student: Athavan k

Annexure A

Design Task

1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support largescale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%
Criteria		Excellent (5)	Good (4)	Average (3)		Poor (2)		
Problem Understanding & Requirement Analysis		Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding		Unclear or incorrect		
System Architecture Design		Complete, scalable, wellstructured architecture with diagrams	Good design with minor issues	Basic design, lacks detail		Poor/incomplete		
Use of PySpark & RDD Operations		Efficient and correct use of RDD transformation s/actions	Mostly correct usage	Limited or partial use		Incorrect/missing		
Intelligent Agent Integration		Strong integration with clear agent roles and logic	Good integration	Basic integration		Poor/no integration		

Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
--	---	---------------------	-----------------------	---------------

Data Flow & Processing Pipeline	Clear, logical, and welldefined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, wellorganized report	Mostly clear	Somewhat organized	Poor presentation

Question 1: Hybrid AI Recommendation Engine Design

1. Problem Understanding & Requirement Analysis

Objective: Build a recommendation engine that suggests items (products, videos, courses) to users by combining content-based filtering (item-attribute similarity) with collaborative filtering (user-behaviour similarity), so that the system works even for new items (cold-start via content) and improves as more interaction data arrives (collaborative signal).

Constraints: The interaction log (clicks, ratings, purchases) can run into hundreds of millions of records for a large platform, so the similarity/matrix computations cannot fit on a single machine. The system must also update recommendations in near real time as a user browses, not just via nightly batch jobs.

Key requirements: (a) distributed storage and processing of user-item interaction data, (b) a hybrid scoring model that fuses two recommendation signals, (c) intelligent agents that continuously adapt to feedback, and (d) low-latency serving of the final ranked list to the application layer.

2. System Architecture Design

The architecture is organised into four layers: a data-ingestion layer, a Spark/RDD processing layer that runs the two filtering pipelines in parallel, an agent layer that fuses and adapts the recommendations, and a serving layer.

Data sources — the clickstream/ratings log, the item catalog (metadata, tags, description text), and a live event stream (Kafka) — are ingested into HDFS/Spark. Two parallel RDD pipelines are built: a Collaborative-Filtering pipeline (user-item rating matrix → Alternating Least Squares) and a Content-Based pipeline (item text/attributes → TF-IDF vectors → cosine similarity). Their outputs are fused by a weighted join into a single ranked list, which a Recommendation Agent serves to the user-facing API, while a Feedback-Learning Agent watches clicks/skips and feeds correction signals back into the scoring weights.

Hybrid AI Recommendation Engine — System Architecture

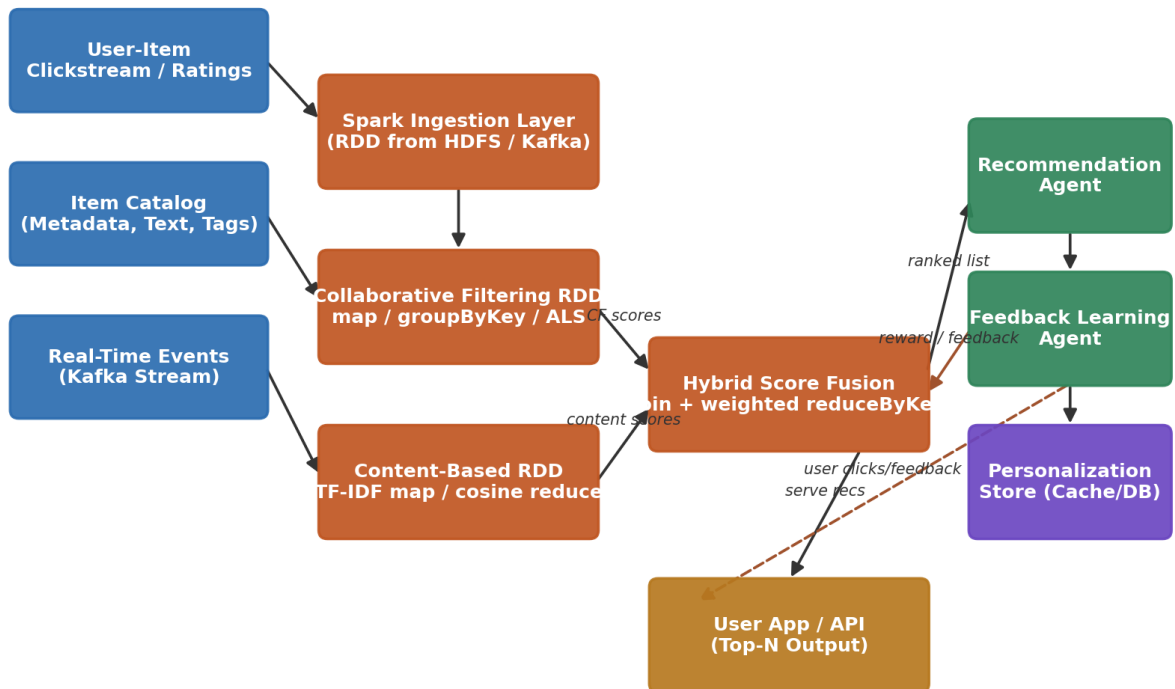


Figure 1: Hybrid AI Recommendation Engine Design — Distributed System Architecture

3. Use of PySpark & RDD Operations

The pipeline is expressed natively as RDD transformations and actions so that it scales horizontally across the Spark cluster:

```

# Load raw interaction logs and item metadata as RDDs
ratings_rdd = sc.textFile("hdfs:///data/ratings.csv") \
    .map(lambda line: line.split(",")) \
    .map(lambda x: (int(x[0]), int(x[1]), float(x[2]))) # (userId, itemId, rating)

items_rdd = sc.textFile("hdfs:///data/items.csv") \
    .map(lambda line: line.split(", ", 2)) \
    .map(lambda x: (int(x[0]), x[2])) # (itemId, description)

# ---- Collaborative Filtering branch ----
user_item_pairs = ratings_rdd.map(lambda r: (r[0], (r[1], r[2])))
user_ratings = user_item_pairs.groupByKey().mapValues(list) # per-user rating vector
cf_model = ALS.train(ratings_rdd.map(lambda r: Rating(*r)), rank=10, iterations=10)

```

```

cf_scores = cf_model.predictAll(candidate_pairs_rdd) \
    .map(lambda r: ((r.user, r.product), r.rating))

# ---- Content-Based branch ----
tfidf_rdd = items_rdd.map(lambda x: (x[0], tokenize(x[1]))) \
    .flatMap(lambda x: (((x[0], term), 1) for term in x[1])) \
    .reduceByKey(lambda a, b: a + b) # term frequency per item
content_scores = tfidf_rdd.map(cosine_similarity_to_profile) # (userId, itemId) ->
score

# ---- Hybrid fusion ----
hybrid_rdd = cf_scores.join(content_scores) \
    .mapValues(lambda s: 0.7 * s[0] + 0.3 * s[1]) # weighted fusion
top_n = hybrid_rdd.map(lambda x: (x[0][0], (x[0][1], x[1]))) \
    .groupByKey().mapValues(lambda v: sorted(v, key=lambda t: -t[1])[0:10])

```

map() extracts and tokenizes fields in parallel across partitions; reduceByKey() and groupByKey() aggregate ratings and term counts per key without shuffling more data than necessary; join() combines the two independently-computed RDDs on (user, item) keys; and the final mapValues/sorted action produces the Top-N list per user as a lazy DAG that Spark only executes when the result is requested, giving fault-tolerant, re-computable, horizontally-scalable processing.

4. Intelligent Agent Integration

- Recommendation Agent: consumes the fused hybrid RDD and applies business rules (diversity, recency, inventory availability) before returning a ranked list — it is the decision-making component that turns raw scores into an actionable, policy-compliant list.
- Feedback-Learning Agent: monitors real-time user actions (click, skip, purchase, dwell time) via the Kafka stream, converts them into an implicit reward signal, and periodically retrains/adjusts the α/β fusion weights and the ALS latent factors — this is what lets the system adapt in real time rather than only on a nightly batch.
- Together the two agents form a simple multi-agent loop: the Recommendation Agent acts, the environment (the user) responds, and the Feedback-Learning Agent

perceives that response and updates the model — a classic sense-decide-act cycle applied at platform scale.

5. Scalability & Distributed Computing Design

- Horizontal scaling: RDD partitions are distributed across worker nodes, so both the CF and content-based pipelines scale by adding executors as the catalog/user base grows.
- Fault tolerance: RDD lineage (the DAG of transformations) lets Spark recompute only the lost partition after a node failure, instead of restarting the whole job.
- Parallelism: the two filtering branches are independent RDD lineages that Spark's scheduler can execute concurrently on different executors before the final join.
- Caching: frequently reused RDDs (e.g., the item TF-IDF vectors, which rarely change) are cached in memory with `.persist()` to avoid recomputation on every request batch.

6. Data Flow & Processing Pipeline

1. Raw events (ratings, clicks, catalog updates) land in HDFS/Kafka.
2. Spark reads them into RDDs and cleans/transforms them (parsing, tokenizing, deduplication).
3. The CF and content-based branches compute similarity/prediction scores in parallel.
4. The hybrid fusion stage joins and re-weights both score sets.
5. The Recommendation Agent packages the Top-N list and pushes it to the serving cache/API.
6. User interactions with the served list flow back through Kafka to the Feedback-Learning Agent, closing the loop.

7. Innovation & Creativity

- Dynamic fusion weighting: instead of a fixed 70/30 CF-content blend, the Feedback-Learning Agent adjusts the weight per user segment — new users lean more on content-based scores (solving cold start), long-tenured users lean more on collaborative scores.
- Context-aware re-ranking: time-of-day, device type, and session intent (browsing vs. buying) are injected as a light re-ranking pass after fusion, adding personalization without retraining the core models.
- Explainable recommendations: because the two branches are kept separate before fusion, the system can tell a user "recommended because it's similar to X" (content) or "because users like you liked it" (collaborative), improving trust.

Question 2: Smart Disaster Detection System

1. Problem Understanding & Requirement Analysis

Objective: Detect natural disasters (floods, earthquakes) as early as possible by fusing three heterogeneous, high-velocity data streams — satellite imagery, ground sensors, and social-media reports — and route verified alerts to the right authorities/communities.

Constraints: Each source has a different format, arrival rate, and reliability (satellite imagery is large but infrequent; sensors are small but continuous; social media is noisy and unstructured). A false negative (missed disaster) is far more costly than a false positive, so the fusion logic must be conservative yet fast.

Key requirements: (a) distributed, low-latency stream ingestion from three heterogeneous sources, (b) a data-fusion layer that reconciles conflicting/partial evidence, (c) intelligent agents that coordinate detection with response planning, and (d) an early-warning broadcast that reaches affected populations within minutes.

2. System Architecture Design

The system uses a lambda-style architecture: a Spark Streaming ingestion layer consumes the three sources continuously as micro-batch RDDs (DStreams), each source is processed by its own specialised RDD pipeline (CNN feature extraction for imagery, threshold/anomaly detection for sensors, NLP sentiment/keyword extraction for social posts), and a Data-Fusion Agent combines the three parallel evidence streams into a single risk score per geographic cell.

When the fused risk score crosses a threshold, an Alert-Coordination Agent hands off to a Response-Planning Agent, which selects the warning channel (SMS, sirens, dashboards) and the affected zone boundary, and pushes the broadcast to end systems.

Smart Disaster Detection — Distributed Multi-Source Architecture

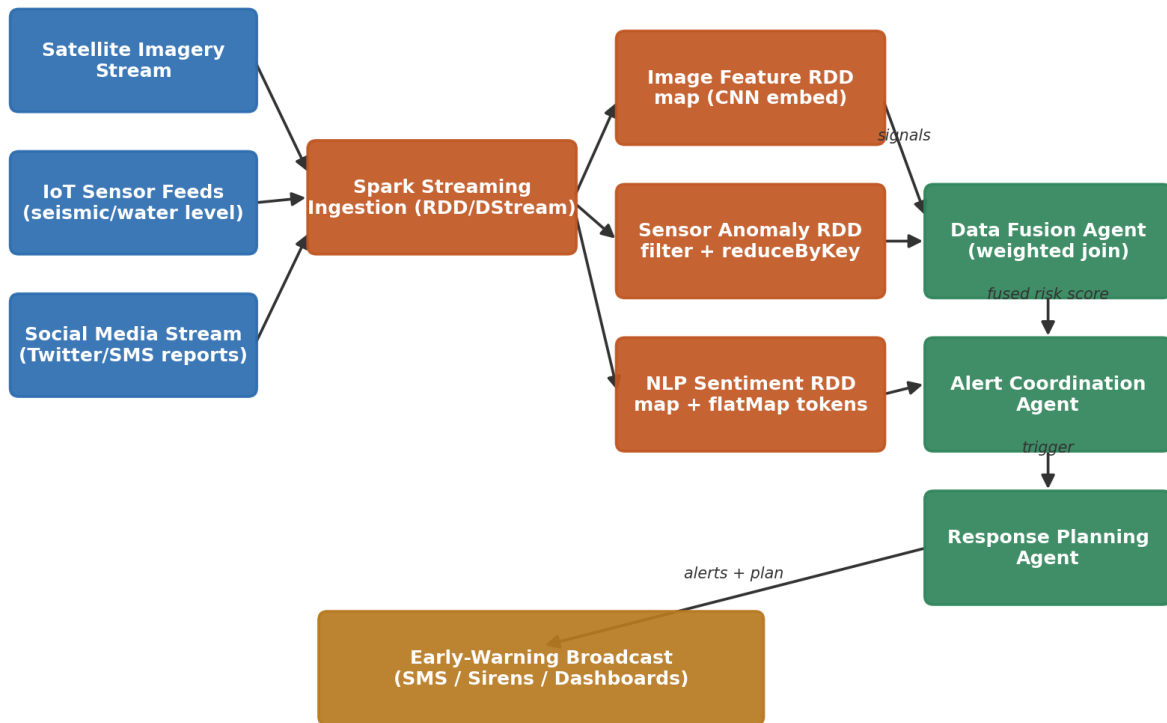


Figure 2: Smart Disaster Detection System — Distributed System Architecture

3. Use of PySpark & RDD Operations

Each source stream is processed as its own RDD/DStream pipeline before fusion:

```

# Satellite imagery stream (feature vectors already extracted per tile)
sat_stream = ssc.socketTextStream("sat-feed", 9001) \
    .map(lambda x: parse_tile(x)) \
    .map(lambda tile: (tile.geo_cell, cnn_embed(tile.image))) # (cell, feature_vector)

# Sensor stream: seismic + water-level readings
sensor_stream = ssc.socketTextStream("sensor-feed", 9002) \
    .map(lambda x: parse_reading(x)) \
    .filter(lambda r: r.value > r.baseline_threshold) \      # anomaly filter
    .map(lambda r: (r.geo_cell, r.severity))

# Social-media stream: keyword + sentiment extraction
social_stream = ssc.socketTextStream("social-feed", 9003) \
    .flatMap(lambda post: extract_disaster_keywords(post)) \
    .map(lambda kw: (kw.geo_cell, kw.urgency_score)) \

```

```

.reduceByKey(lambda a, b: a + b)

# ---- Fusion (per micro-batch) ----
def fuse(rdd_batch):
    sat, sen, soc = rdd_batch
    fused = sat.join(sen).join(soc) \
        .mapValues(lambda v: 0.4*v[0][0] + 0.4*v[0][1] + 0.2*v[1]) # weighted risk
    score
    alerts = fused.filter(lambda x: x[1] > ALERT_THRESHOLD)
    return alerts

alerts_rdd = fuse((sat_stream, sensor_stream, social_stream))
alerts_rdd.foreachRDD(lambda rdd: dispatch_alert_agent(rdd.collect()))

```

filter() discards sub-threshold sensor noise before it ever reaches the fusion stage, reducing shuffle volume; reduceByKey() aggregates many noisy social posts per geo-cell into one urgency score; and join() aligns the three independently-processed streams on the shared geo_cell key so the risk score for a location reflects all three evidence types, all computed per micro-batch in a fault-tolerant, restartable DAG.

4. Intelligent Agent Integration

- Data-Fusion Agent: reconciles evidence from imagery, sensors, and social posts, resolving conflicts (e.g., a sensor spike with no corroborating imagery is weighted lower than one with both).
- Alert-Coordination Agent: decides when the fused risk score is strong enough to escalate, avoiding both alert fatigue (too many false alarms) and missed events, and coordinates with neighbouring geo-cells so a spreading flood/earthquake is treated as one connected event rather than isolated cells.
- Response-Planning Agent: once an alert is confirmed, selects the appropriate channel and message per zone (e.g., sirens for a fast-onset flash flood vs. SMS/dashboard for a slower-rising river flood) and can suggest evacuation routes based on the affected-area geometry.

5. Scalability & Distributed Computing Design

- Micro-batch streaming: Spark Streaming processes each source as short RDD batches (e.g., every 2 seconds), giving near-real-time latency while retaining RDD fault tolerance.
- Partitioning by geo-cell: keying every stream by geographic cell allows the fusion join to be co-partitioned, minimising network shuffle even as the number of sensors/posts grows into the millions.
- Elastic scaling: during an unfolding disaster, social-media and sensor volume spikes sharply; the cluster can autoscale executors to absorb the surge without dropping data.
- Checkpointing: DStream checkpointing ensures that if a worker fails mid-disaster (the worst possible time to lose data), processing resumes from the last checkpoint rather than restarting.

6. Data Flow & Processing Pipeline

7. Three raw streams (satellite, sensor, social) are ingested concurrently via Spark Streaming.
8. Each stream is cleaned and transformed into a per-geo-cell evidence score.
9. The Data-Fusion Agent joins the three evidence scores into one fused risk score per cell.
10. The Alert-Coordination Agent evaluates the fused score against thresholds and neighbouring cells.
11. The Response-Planning Agent generates the warning content and channel selection.
12. The broadcast layer pushes alerts to SMS gateways, sirens, and public dashboards.

7. Innovation & Creativity

- Cross-validation between sources before alerting: requiring at least two of the three independent sources to agree before triggering a public alert sharply cuts false positives from noisy social media alone.

- Spatial spread modelling: treating adjacent geo-cells as a connected graph lets the system project how a flood/earthquake's affected area will grow over the next few minutes, giving a head start to response planning.
- Confidence-tiered alerting: a low-confidence fused score triggers a quiet notification to authorities for verification, while a high-confidence score triggers the full public broadcast — a graduated response instead of a binary alarm.

Question 3: AI-Based Fraud Detection Framework

1. Problem Understanding & Requirement Analysis

Objective: Detect fraudulent financial transactions among very large transaction volumes in near real time, minimising both false negatives (missed fraud) and false positives (blocking legitimate customers).

Constraints: Transaction volume can reach millions of records per hour for a large bank/payment processor; fraud patterns evolve constantly (adversarial adaptation), so a static rule set quickly becomes stale; and decisions on card transactions must be made in milliseconds to seconds.

Key requirements: (a) distributed processing of large transaction volumes, (b) anomaly-detection algorithms implemented in a scalable, parallel manner, (c) intelligent agents that collaborate to score, escalate, and continuously retrain, and (d) an audit-friendly case-management pathway for human analysts.

2. System Architecture Design

Incoming transactions from the payment gateway and the historical ledger (HDFS) are ingested into a Spark RDD pipeline that extracts behavioural features (transaction velocity, deviation from the customer's normal spend pattern, geolocation jump, merchant risk category). Two parallel scoring paths run: a statistical/ML anomaly-detection RDD (e.g., Isolation Forest style partition scoring) and a deterministic rule-engine RDD (hard business rules such as impossible-travel or blacklisted merchant).

A Detection Agent merges both scores into a final risk score per transaction; an Alert & Escalation Agent routes high-risk transactions to a block/hold API and low/medium-risk ones to an analyst dashboard; and a Model-Update Agent continuously folds analyst-confirmed outcomes back into the anomaly model so detection improves over time.

AI-Based Fraud Detection Framework — Architecture

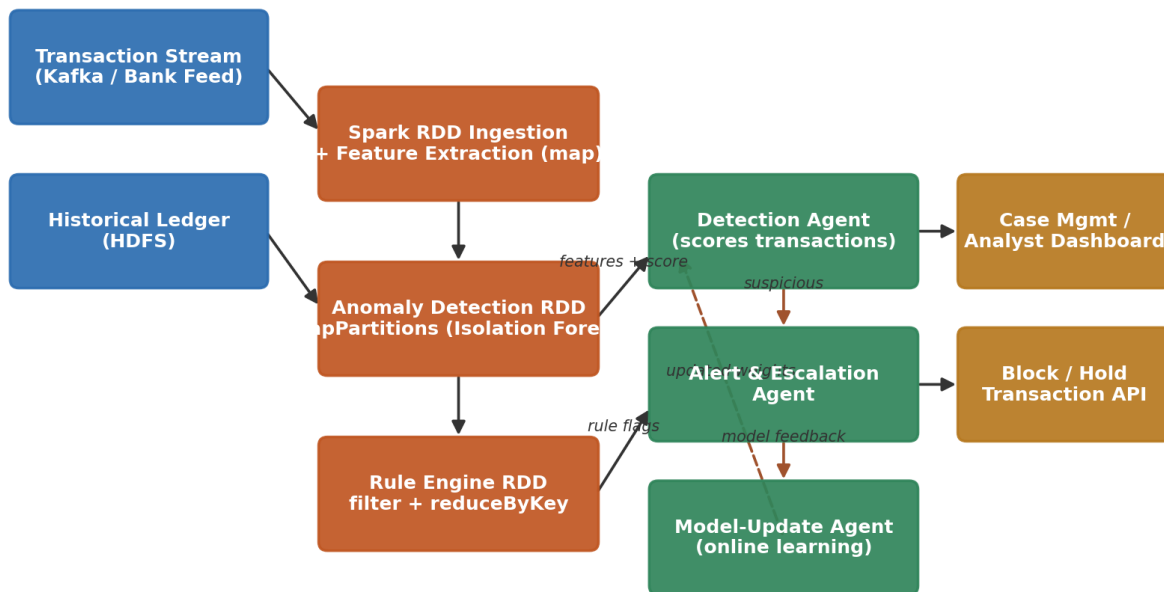


Figure 3: AI-Based Fraud Detection Framework — Distributed System Architecture

3. Use of PySpark & RDD Operations

The scoring pipeline splits into a statistical branch and a rule branch, executed as RDD transformations for horizontal scale:

```

txn_rdd = sc.textFile("hdfs:///data/transactions/*.csv") \
    .map(lambda line: parse_txn(line))          # -> Transaction objects

# Feature extraction, keyed by account id for locality
features_rdd = txn_rdd.map(lambda t: (t.account_id, t)) \
    .groupByKey() \
    .flatMapValues(lambda txns: compute_behavioural_features(txns))

# ---- Statistical anomaly-detection branch ----
def score_partition(iterator):
    model = load_isolation_forest_model()      # loaded once per partition
    for account_id, feat_vec in iterator:
        yield (account_id, model.score(feat_vec))
anomaly_scores = features_rdd.mapPartitions(score_partition)

```

```
# ---- Deterministic rule-engine branch ----
rule_flags = txn_rdd.filter(lambda t: violates_rule(t)) \
    .map(lambda t: (t.account_id, 1)) \
    .reduceByKey(lambda a, b: a + b)          # count of rule violations per account

# ---- Fusion into final risk score ----
risk_rdd = anomaly_scores.leftOuterJoin(rule_flags) \
    .mapValues(lambda v: v[0] + 0.5 * (v[1] or 0))
high_risk = risk_rdd.filter(lambda x: x[1] > RISK_THRESHOLD)
high_risk.foreachPartition(lambda part: block_and_alert(list(part)))
```

mapPartitions() loads the trained anomaly model once per partition (not once per record), which is essential for performance at scale; filter() isolates rule violations without touching unrelated records; reduceByKey() tallies rule breaches per account in parallel; and leftOuterJoin() safely fuses statistical and rule-based evidence even when one branch has no data for a given account, all within Spark's fault-tolerant DAG execution.

4. Intelligent Agent Integration

- Detection Agent: fuses the statistical anomaly score and the rule-violation count into one risk score and applies the decision threshold — the core "is this fraud?" judgement.
- Alert & Escalation Agent: routes transactions by severity — auto-block for very high risk, hold-and-notify-customer for medium risk, and log-for-review for low risk — coordinating with the case-management dashboard so analysts see a prioritised queue rather than a raw firehose.
- Model-Update Agent: collects analyst verdicts (confirmed fraud / false alarm) and periodically retrains or fine-tunes the anomaly-detection model, closing the loop so the system adapts as fraud patterns shift — this is the collaborative, continuously-improving element the rubric calls for.

5. Scalability & Distributed Computing Design

- Data-local partitioning by account id keeps each account's transaction history co-located, avoiding expensive shuffles during feature extraction.
- mapPartitions for model scoring amortises model-loading cost across many records per partition, which is critical when scaling to millions of transactions per hour.
- Fault tolerance via RDD lineage means a failed executor during a scoring run only requires recomputing its partitions, not the whole batch — important when transaction scoring cannot be allowed to silently drop records.
- Elastic scaling for peak periods (e.g., holiday shopping surges) by adding executors, since the RDD pipeline parallelises linearly with the number of partitions.

6. Data Flow & Processing Pipeline

13. Transactions stream in from the payment gateway and are appended to the historical ledger.
14. Spark extracts behavioural features per account from the combined stream.
15. The statistical and rule-based branches score transactions in parallel.
16. The Detection Agent fuses both scores into a final risk value.
17. The Alert & Escalation Agent routes the transaction to block, hold, or review.
18. Analyst decisions flow back to the Model-Update Agent, which retrain the anomaly model.

7. Innovation & Creativity

- Graph-based collusion detection: extending the account-level features with a transaction graph (shared devices, IPs, or beneficiaries across accounts) to catch coordinated fraud rings that a single-account view would miss.
- Adaptive thresholding: instead of a fixed risk threshold, the Detection Agent adjusts the cut-off per merchant category and time-of-day based on recent false-positive rates, balancing fraud capture against customer friction.
- Explainable risk scores: surfacing the specific rule violations and top contributing features alongside the score, so analysts (and regulators) can audit why a transaction was flagged.

Question 4: Autonomous Supply Chain Optimization System

1. Problem Understanding & Requirement Analysis

Objective: Optimise inventory levels, logistics routing, and demand forecasting across a distributed supply chain (multiple warehouses, transport routes, and suppliers) to minimise cost while meeting service-level targets.

Constraints: Data arrives from many independent sources (point-of-sale systems, GPS-tracked fleets, weather/market feeds) at different warehouses/regions; decisions at one node (e.g., a warehouse reducing stock) affect others (e.g., a logistics route needing rebalancing), so the system must support decentralised, coordinated decision-making rather than one central bottleneck planner.

Key requirements: (a) large-scale distributed processing of multi-source supply-chain data, (b) RDD-based transformations for forecasting, inventory optimisation, and route planning, and (c) multiple intelligent agents that negotiate and act semi-independently to minimise overall cost.

2. System Architecture Design

POS/inventory data, logistics/GPS feeds, and market/weather signals are ingested into a Spark RDD cleaning layer. Three parallel RDD pipelines then run: demand forecasting (time-series modelling per SKU/region), inventory optimisation (economic-order-quantity style computation per warehouse), and route optimisation (vehicle routing computation per logistics hub).

Each pipeline's output feeds a corresponding agent — a Warehouse Agent (stocking decisions), a Logistics Agent (routing decisions), and a Negotiation/Coordinator Agent that resolves conflicts between warehouses and routes (e.g., when reducing stock at Warehouse A means Logistics must reroute deliveries from Warehouse B) — producing a final decentralised, cost-minimising plan.

Autonomous Supply Chain Optimization — Architecture

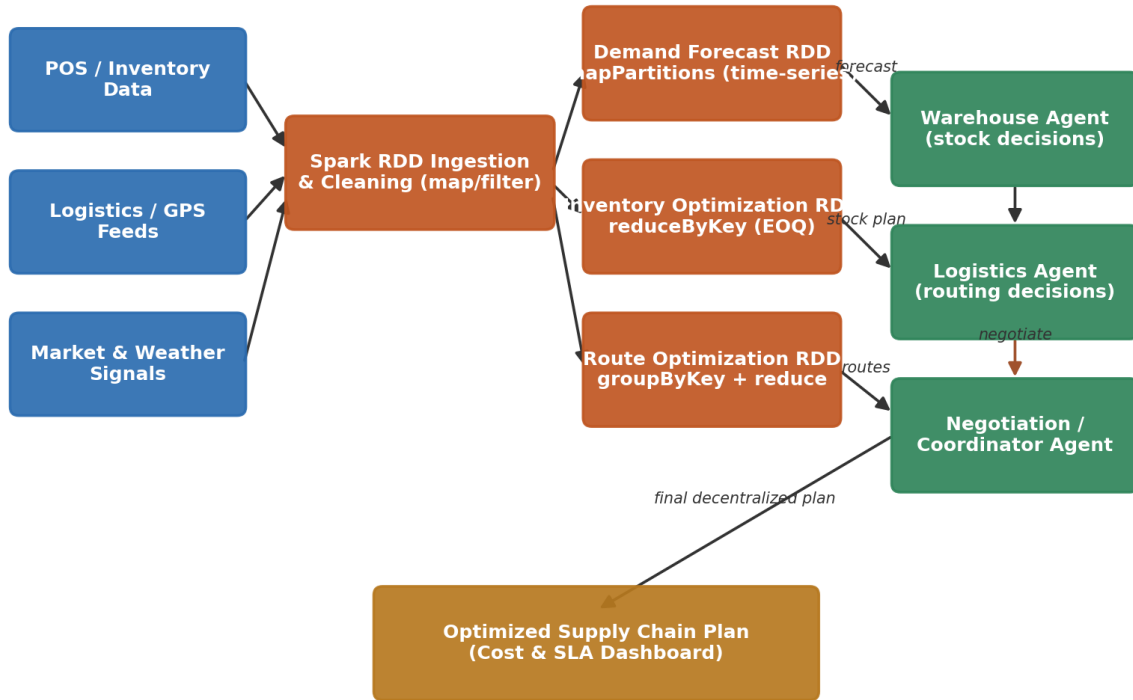


Figure 4: Autonomous Supply Chain Optimization System — Distributed System Architecture

3. Use of PySpark & RDD Operations

The three optimisation pipelines are expressed as parallel RDD transformation chains:

```

sales_rdd = sc.textFile("hdfs:///data/pos/*.csv").map(parse_sale)      # (sku, region,
qty, ts)
gps_rdd = sc.textFile("hdfs:///data/gps/*.csv").map(parse_gps)      # (vehicle,
route, ts, loc)
signals_rdd = sc.textFile("hdfs:///data/signals/*.csv").map(parse_signal) # (region,
weather, market_idx)

# ---- Demand forecasting branch ----
sku_series = sales_rdd.map(lambda s: ((s.sku, s.region), (s.ts, s.qty))) \
    .groupByKey().mapValues(lambda v: sorted(v))
forecast_rdd = sku_series.mapPartitions(lambda it: run_timeseries_model(it)) # ->
(sku, region, forecast_qty)

# ---- Inventory optimisation branch (RDD transformations) ----
inventory_rdd = forecast_rdd.map(lambda f: (f.region, compute_eoq(f))) \

```

```

    .reduceByKey(lambda a, b: merge_orders(a, b))          # aggregate reorder plan
per warehouse

# ---- Route optimisation branch ----
route_rdd = gps_rdd.map(lambda g: (g.route, g)) \
    .groupByKey() \
    .mapValues(lambda stops: optimise_route(list(stops)))    # nearest-neighbour /
VRP heuristic

# ---- Decentralised negotiation between agents ----
plan_rdd = inventory_rdd.join(route_rdd.map(lambda r: (route_to_region(r[0]), r[1])))
\
    .mapValues(lambda v: negotiate_plan(v[0], v[1]))        # coordinator agent logic
plan_rdd.saveAsTextFile("hdfs:///output/supply_chain_plan")

```

groupByKey() assembles each SKU's or route's full history so time-series/routing algorithms have complete context per key; mapPartitions() runs the (relatively expensive) forecasting and routing models once per partition for efficiency; reduceByKey() merges reorder recommendations per warehouse region; and the final join() lets the independently-computed inventory and route plans be reconciled by the coordinator logic — mirroring how real warehouses and logistics teams negotiate rather than being centrally dictated to.

4. Intelligent Agent Integration

- Warehouse Agent: owns stocking decisions for its warehouse, using the demand forecast and EOQ output to decide what and how much to reorder, and can request reallocation from neighbouring warehouses instead of a full supplier order when cheaper.
- Logistics Agent: owns routing decisions, optimising vehicle loads and delivery sequences based on current GPS data, fuel cost, and delivery deadlines.
- Negotiation/Coordinator Agent: mediates between Warehouse and Logistics agents when their locally optimal decisions conflict (e.g., a warehouse wants to destock quickly, but that requires an extra logistics trip that raises overall cost) — it performs a lightweight multi-agent negotiation (compare marginal cost of each option) to reach a jointly better decision, which is the essence of decentralised decision-making for cost minimisation described in the brief.

5. Scalability & Distributed Computing Design

- Regional partitioning: keying RDDs by region/warehouse lets each region's forecasting, inventory, and routing computation run independently and in parallel, scaling naturally as more warehouses are added.
- Fault tolerance: RDD lineage allows recovery of a lost partition (e.g., one region's route computation) without re-running the entire network's plan.
- Incremental recomputation: only regions with materially changed data (new sales, rerouted vehicles) need their RDD lineage recomputed, keeping the system responsive as it scales to a national/global supply chain.
- Decentralised agents: because each Warehouse/Logistics Agent works from its own regional RDD, the design avoids a single central planner becoming a bottleneck as the number of nodes grows.

6. Data Flow & Processing Pipeline

19. POS, GPS, and market/weather data are ingested and cleaned as RDDs.
20. The demand-forecasting pipeline produces per-SKU, per-region forecasts.
21. The inventory-optimisation pipeline turns forecasts into reorder plans per warehouse.
22. The route-optimisation pipeline turns GPS/order data into optimised delivery routes.
23. The Warehouse and Logistics Agents propose their locally optimal decisions.
24. The Coordinator Agent negotiates conflicts and outputs the final decentralised supply-chain plan.

7. Innovation & Creativity

- Marginal-cost negotiation protocol: agents exchange a simple "cost of my proposed action" number rather than raw data, keeping negotiation lightweight and scalable across many warehouses.
- Weather/market-aware forecasting: folding weather and market-index signals directly into the time-series model lets the system anticipate demand spikes (e.g., a heatwave driving beverage demand) before they show up in sales data.

- What-if simulation mode: the same RDD pipeline can be re-run against hypothetical scenarios (a supplier delay, a route closure) to let planners stress-test the supply chain before committing to a plan.

Question 5: Personalized Learning Analytics Platform

1. Problem Understanding & Requirement Analysis

Objective: Analyse learning-behaviour and performance data for thousands of students to personalise content delivery, support adaptive learning paths, and give real-time feedback to both students and instructors.

Constraints: Data volume grows continuously as students interact with the LMS (clicks, video watch time, quiz attempts, forum posts) across potentially thousands of concurrent learners; personalisation must be timely enough to influence the student's current session, not just a nightly report.

Key requirements: (a) scalable analytics over large, continuously-growing behavioural datasets, (b) RDD-based processing for learner profiling and performance prediction, (c) intelligent agents that adapt content delivery and give real-time feedback, and (d) a dashboard that personalises at the scale of thousands of learners simultaneously.

2. System Architecture Design

LMS clickstream data, historical assessment scores, and engagement signals (time-on-task, forum activity) are ingested into a Spark RDD layer. Three parallel RDD pipelines process this data: Learner Profiling (aggregating mastery level per concept per student), Performance Prediction (a model estimating likely outcomes on upcoming assessments), and Content Similarity (mapping concepts to related learning resources via a concept graph).

A Tutoring Agent uses the learner profile, prediction, and content-similarity outputs to select the next best learning activity for each student (adaptive learning path), while a Feedback Agent monitors the student's live session and pushes real-time nudges (hints, encouragement, remediation prompts) — both feeding a personalised dashboard for students and instructors.

Personalized Learning Analytics Platform — Architecture

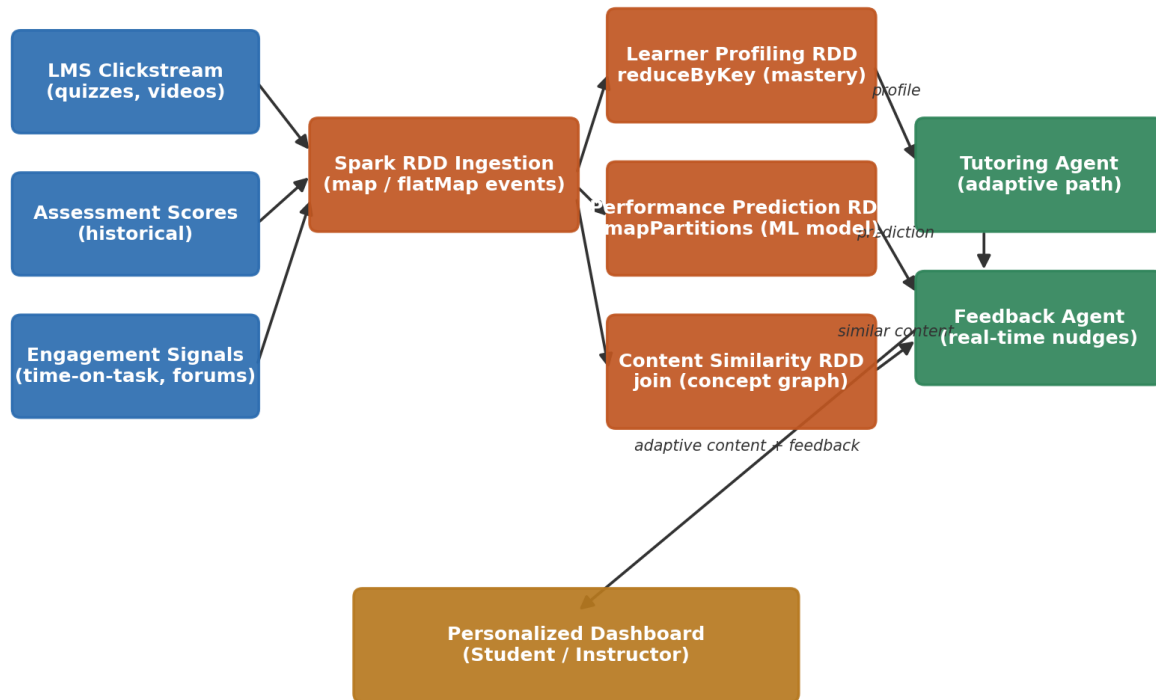


Figure 5: Personalized Learning Analytics Platform — Distributed System Architecture

3. Use of PySpark & RDD Operations

The profiling, prediction, and content-matching pipelines run as parallel RDD transformations:

```

events_rdd = sc.textFile("hdfs:///data/lms_events/*.json") \
    .flatMap(lambda line: parse_event(line))          # -> (student_id, concept,
event_type, score)

# ---- Learner profiling branch ----
concept_scores = events_rdd.map(lambda e: ((e.student_id, e.concept), e.score)) \
    .reduceByKey(lambda a, b: (a + b) / 2)           # running mastery estimate
profile_rdd = concept_scores.map(lambda kv: (kv[0][0], (kv[0][1], kv[1]))) \
    .groupByKey().mapValues(dict)                    # student -> {concept: mastery}

# ---- Performance prediction branch ----
def predict_partition(iterator):
    model = load_prediction_model()
    for student_id, profile in iterator:

```



```

    yield (student_id, model.predict(profile))
prediction_rdd = profile_rdd.mapPartitions(predict_partition)

# ---- Content similarity branch ----
weak_concepts = profile_rdd.flatMapValues(lambda p: [c for c, m in p.items() if m <
MASTERY_THRESHOLD])
recommended_content = weak_concepts.map(lambda x: (x[1], x[0])) \
    .join(concept_resource_rdd)                # concept -> resource catalog
    .map(lambda x: (x[1][0], x[1][1]))        # (student_id, resource)

# ---- Adaptive path assembly ----
adaptive_path =
profile_rdd.join(prediction_rdd).join(recommended_content.groupByKey()) \
    .mapValues(lambda v: build_learning_path(v))
adaptive_path.saveAsTextFile("hdfs:///output/adaptive_paths")

```

`flatMap()` unpacks each raw LMS event log into individual (student, concept, score) records; `reduceByKey()` maintains a running per-concept mastery estimate without needing the full history in memory at once; `mapPartitions()` amortises the prediction model's load cost across many students per partition; and `join()/groupByKey()` bring together profile, prediction, and recommended-content RDDs into one adaptive learning path per student, all executed lazily and re-computably across the cluster.

4. Intelligent Agent Integration

- Tutoring Agent: reads the learner's profile, predicted outcome, and matched content to decide the next best activity — e.g., moving a student ahead if mastery is strong, or inserting a remediation resource if a concept is weak — the core adaptive-learning decision-maker.
- Feedback Agent: watches the live session stream (current quiz attempt, video pause points) and pushes immediate, low-latency nudges such as a hint after repeated wrong answers or an encouragement message after a good streak, closing the loop between the batch-level profile and the student's current moment.
- The two agents divide responsibility naturally: the Tutoring Agent handles medium-term path planning (what to study next), while the Feedback Agent handles moment-

to-moment support (how to help right now) — together giving both strategic and immediate personalisation.

5. Scalability & Distributed Computing Design

- Partitioning by student id lets the profiling and prediction pipelines scale linearly as the learner base grows into the thousands, since each partition's work is independent.
- `mapPartitions` for the prediction model avoids reloading the model per student, which matters when scoring thousands of learners in each analytics cycle.
- Incremental updates: `reduceByKey`-based running mastery estimates mean new events update the profile incrementally rather than requiring a full recompute over the student's entire history.
- Fault tolerance via RDD lineage ensures that a lost partition (e.g., one batch of a cohort's events) can be recomputed without disrupting the whole platform's analytics run.

6. Data Flow & Processing Pipeline

25. LMS clickstream, assessment scores, and engagement signals are ingested as RDDs.
26. The learner-profiling pipeline aggregates per-concept mastery per student.
27. The performance-prediction pipeline estimates likely outcomes from the profile.
28. The content-similarity pipeline matches weak concepts to remedial/next resources.
29. The Tutoring Agent assembles an adaptive learning path from all three outputs.
30. The Feedback Agent layers real-time nudges on top during the live session, both surfaced on the personalised dashboard.

7. Innovation & Creativity

- Peer-similarity grouping: clustering students with similar mastery-gap patterns lets the platform recommend collaborative study groups, not just individual content — adding a social learning dimension.
- Confidence-aware prediction: the Performance Prediction branch outputs a confidence interval, not just a point estimate, so the Tutoring Agent can be more conservative (offer more practice) when the model is uncertain about a student.

- Instructor-facing early-warning view: the same profile/prediction RDDs are aggregated at the class level to flag students trending toward disengagement, giving instructors a scalable way to intervene across thousands of learners without manually reviewing each one.